

# Chapter 5

## Meta-heuristic

Meta-heuristic methods are high-level, problem-independent frameworks that guide underlying search procedures to explore large solution spaces effectively, trading guaranteed optimality for practical scalability. This chapter presents four meta-heuristic algorithms applied to the multi-team task scheduling problem:

- **Genetic Algorithm (GA)**
- **Simulated Annealing (SA)**
- **Tabu Search (TS)**
- **Ant Colony Optimization (ACO)**

Despite their different search paradigms, all four methods share a common algorithmic backbone: a unified solution representation, the same greedy initialization, an identical decode-and-evaluate pipeline, and the same basic perturbation operators. These shared components are defined once in Section 5.1 and reused across the subsequent method-specific sections.

### 5.1 Shared Algorithmic Components

The following building blocks appear identically (or near-identically) in all four meta-heuristic implementations and are therefore defined here to avoid repetition.

#### 5.1.1 Schedulable Order Computation

Before any search begins, a fixed topological ordering of *schedulable* tasks is computed. A task is schedulable if at least one team is capable of performing it (`can_do[i] = 1`). Kahn's BFS-based topological sort is applied; only schedulable tasks are retained in the final ordering `schedulable_order`, which is then kept fixed throughout the entire search.

---

**Algorithm 3** Compute Schedulable Order

---

```

1: function COMPUTE_SCHEDULABLE_ORDER(num_task, precedence_graph,
   can_do)
2:   in_deg  $\leftarrow$   $[0] \times \text{num\_task}$ 
3:   for  $u \leftarrow 0$  to num_task - 1 do
4:     for  $v \in \text{precedence\_graph}[u]$  do
5:       in_deg[v] += 1
6:     end for
7:   end for
8:   queue  $\leftarrow$  deque( $\{i \mid \text{in\_deg}[i] = 0\}$ )
9:   full_order  $\leftarrow$  []
10:  while queue is not empty do
11:     $u \leftarrow \text{queue.popleft}()$ 
12:    full_order.append( $u$ )
13:    for  $v \in \text{precedence\_graph}[u]$  do
14:      in_deg[v] -= 1
15:      if in_deg[v] = 0 then
16:        queue.append( $v$ )
17:      end if
18:    end for
19:  end while
20:  return [ $u \mid u \in \text{full\_order}, \text{can\_do}[u] = 1$ ]
21: end function

```

---

### 5.1.2 Solution Representation

Each candidate solution is encoded as a pair of equal-length lists:

- **task\_order**: a topologically valid permutation of schedulable tasks, specifying the order in which tasks are processed.
- **team\_assignment**: the  $k$ -th entry is the team assigned to the  $k$ -th task in **task\_order**.

**Structure:**

State / Chromosome / Solution:

```

{
  "task_order"      : [3, 1, 4, 2, 5],
  "team_assignment" : [2, 1, 3, 1, 2]
}

```

**Example:** The above encodes that tasks are executed in order  $3 \rightarrow 1 \rightarrow 4 \rightarrow 2 \rightarrow 5$ ; task 3 is done by team 2, task 1 by team 1, task 4 by team 3, task 2 by team 1, and task 5 by team 2.

### 5.1.3 Initial Solution Construction

All four methods initialise their search from a *greedy* solution. The greedy constructor processes tasks in the fixed schedulable order and, for each task, selects the team whose earliest available start time is smallest (ties broken by cost).

---

#### Algorithm 4 Greedy Construction

---

```

1: function GREEDY_CONSTRUCT(schedulable_order, rev_graph, team_avail,
   duration, cost)
2:   task_End  $\leftarrow [0] \times \text{num\_task}$ 
3:   teamAvail  $\leftarrow \text{team\_avail}[:]$ 
4:   task_order  $\leftarrow []$ ; team_assignment  $\leftarrow []$ 
5:   for currentTask  $\in$  schedulable_order do
6:     pred_finish  $\leftarrow 0$ 
7:     for  $p \in \text{rev\_graph}[\text{currentTask}]$  do
8:       if task_End[p] > pred_finish then
9:         pred_finish  $\leftarrow \text{task\_End}[p]$ 
10:      end if
11:    end for
12:    best_choice  $\leftarrow \text{None}$ ; best_time  $\leftarrow \infty$ ; best_cost  $\leftarrow \infty$ 
13:    for (team, c)  $\in$  cost[currentTask] do
14:      earliest  $\leftarrow \max(\text{teamAvail}[\text{team}], \text{pred\_finish})$ 
15:      if earliest < best_time or (earliest = best_time and c < best_cost)
16:        then
17:          best_time  $\leftarrow$  earliest; best_cost  $\leftarrow$  c; best_choice  $\leftarrow$  team
18:        end if
19:      end for
20:      task_order.append(currentTask); team_assignment.append(best_choice)
21:      task_End[currentTask]  $\leftarrow$  best_time + duration[currentTask]
22:      teamAvail[best_choice]  $\leftarrow$  task_End[currentTask]
23:    end for
24:  return State(task_order, team_assignment)
25: end function

```

---

### 5.1.4 Decode Solution and Fitness Function

Given a solution, the decoder simulates task execution according to **task\_order**. For each task, the actual start time is determined by the earliest feasible time after considering the assigned team's availability, the completion time of all predecessors, and the finish time of the previous task assigned to the same team.

Since the problem is multi-objective, *lexicographic* fitness triple is used across all four methods:

$$\text{fitness} = \underbrace{(M - \text{scheduled})}_{\text{minimize}}, \underbrace{\text{completion}}_{\text{minimize}}, \underbrace{\text{total cost}}_{\text{minimize}}$$

The fitness function evaluates the quality of each solution by calculating three main criteria:

- **Number of unscheduled tasks:** The number of tasks that cannot be scheduled. This criterion aims to minimize the number of unscheduled tasks, or equivalently maximize the total number of scheduled tasks.
- **Completion time:** The overall completion time of the whole schedule. This criterion aims to minimize the total execution time, also known as the makespan.
- **Assigning cost:** The overall assignment cost of all scheduled tasks. This criterion aims to minimize the total cost.

Tuples are compared lexicographically: fewer unscheduled tasks are preferred first, then smaller completion time, and finally lower total cost. The decoding procedure is described as follows:

1. Simulate task execution in **task\_order**.
2. For each task:
  - Find the **earliest feasible start time** after:
    - the assigned team's availability  $s(j)$ ;
    - the completion time of all predecessor tasks;
    - the finish time of the previous task assigned to the same team.
  - If feasible, schedule the task and compute:
    - start time;
    - end time;
    - assignment cost  $c(i, j)$ .
3. Finally, keep track of:
  - total number of scheduled tasks;
  - completion time;
  - total cost.

---

#### Algorithm 5 Decode Solution

---

```

1: function DECODE_SOLUTION(sol, duration, num_task, team_avail, rev_graph)
2:   task_End  $\leftarrow [0] \times \text{num\_task}$ ;   teamAvail  $\leftarrow \text{team\_avail}[:]$ 
3:   schedule  $\leftarrow []$ 
4:   for (currentTask, team)  $\in \text{zip}(\text{sol.task\_order}, \text{sol.team\_assignment})$  do
5:     pred_finish  $\leftarrow 0$ 
6:     for  $p \in \text{rev\_graph}[\text{currentTask}]$  do
7:       if task_End[p] > pred_finish then
8:         pred_finish  $\leftarrow \text{task\_End}[p]$ 
9:       end if
10:    end for
11:    earliest  $\leftarrow \max(\text{teamAvail}[\text{team}], \text{pred\_finish})$ 
12:    teamAvail[team]  $\leftarrow \text{earliest} + \text{duration}[\text{currentTask}]$ 

```

---

```

13:     task_End[currentTask] ← earliest + duration[currentTask]
14:     schedule.append((currentTask, team, earliest))
15:   end for
16:   return schedule
17: end function

```

---



---

**Algorithm 6** Calculate Fitness

---

```

1: function CALC_FITNESS(sol, num_task, cost, duration, team_avail, rev_graph)
2:   schedule ← DECODE_SOLUTION(sol, duration, num_task, team_avail,
   rev_graph)
3:   finish_task ← 0; completion ← 0; totalCost ← 0
4:   for (task, team, start) ∈ schedule do
5:     finish_task += 1
6:     finish ← start + duration[task]
7:     if finish > completion then
8:       completion ← finish
9:     end if
10:    totalCost += cost[task][team]
11:  end for
12:  return (num_task - finish_task, completion, totalCost)
13: end function

```

---

### 5.1.5 Precedence Feasibility Check

Before any permutation of tasks is accepted as a valid ordering, it must satisfy all precedence constraints.

---

**Algorithm 7** Precedence Feasibility Check

---

```

1: function IS_PRECEDENCE_FEASIBLE(task_order, precedence)
2:   pos ← {task : idx | (idx, task) ∈ enumerate(task_order)}
3:   for (i, j) ∈ precedence do
4:     if i ∈ pos and j ∈ pos and pos[i] ≥ pos[j] then
5:       return False
6:     end if
7:   end for
8:   return True
9: end function

```

---

### 5.1.6 Neighbor Generation

All search-based methods perturb a current solution via two complementary operators applied in sequence.

## Task Order Mutation

Two positions  $i, j$  are randomly chosen and swapped. The swap is accepted only if the resulting order remains precedence-feasible; otherwise the attempt is retried up to `attempts` times. The corresponding team entries are swapped in tandem.

---

### Algorithm 8 Task Order Mutation

---

```
1: function TASK_ORDER_MUTATION(task_order, team_assignment, precedence,
   attempts)
2:   if |task_order| < 2 then
3:     return task_order[:]
4:   end if
5:   for  $k \leftarrow 1$  to attempts do
6:     new_order  $\leftarrow$  task_order[:]
7:      $i, j \leftarrow$  random sample of 2 distinct indices from new_order
8:     swap(new_order[i], new_order[j])
9:     if IS_PRECEDENCE_FEASIBLE(new_order, precedence) then
10:      swap(team_assignment[i], team_assignment[j])
11:      return new_order
12:   end if
13: end for
14: return task_order[:]
15: end function
```

---

## Team Assignment Mutation

Each task is independently re-assigned to a *different* capable team with probability `mutation_rate`.

---

### Algorithm 9 Team Assignment Mutation

---

```
1: function TEAM_ASSIGNMENT_MUTATION(task_order, team_assignment,
   available_team, mutation_rate)
2:   new_assignment  $\leftarrow$  team_assignment[:]
3:   for (idx, task)  $\in$  enumerate(task_order) do
4:     if random() < mutation_rate then
5:       viable  $\leftarrow$  available_team[task]
6:       if |viable| > 1 then
7:         old_team  $\leftarrow$  new_assignment[idx]; new_team  $\leftarrow$  old_team
8:         while new_team = old_team do
9:           new_team  $\leftarrow$  random choice from viable
10:        end while
11:        new_assignment[idx]  $\leftarrow$  new_team
12:      end if
13:    end if
14:  end for
15:  return new_assignment
```

---

## 5.2 Genetic Algorithm

**Introduction:** Genetic Algorithms (GA) are evolutionary algorithms inspired by natural selection. They maintain a *population* of candidate solutions and improve solution quality across generations by selecting fit individuals, combining their traits via crossover, and introducing random mutations.

**Main Idea:** By repeatedly selecting the fittest individuals, mixing their information through crossover, and mutating offspring to maintain diversity, GA balances broad exploration of the search space with focused exploitation of promising regions.

1. **Initialization:** Start with a population of candidate solutions (chromosomes).
2. **Selection:** Choose the best candidates based on a fitness function.
3. **Crossover:** Combine parts of two parents to create new offspring.
4. **Mutation:** Randomly alter some genes to maintain diversity.
5. **Replacement:** Form a new generation and repeat the process.

### 5.2.1 Chromosome Representation

GA uses the solution representation defined in Section 5.1.2: each chromosome consists of a `task_order` list together with its corresponding `team_assignment` list.

### 5.2.2 Population Initialization

The population is seeded with one greedy individual (Section 5.1.3) and (`population_size` - 1) randomly constructed feasible individuals. A random individual is built via a randomised topological sort over the schedulable tasks.

---

#### Algorithm 10 Random Individual Construction

---

```

1: function CONSTRUCT_RANDOM(schedulable_order, precedence_graph,
   available_team)
2:   task_set  $\leftarrow$  set(schedulable_order)
3:   Compute in_deg[u] for all  $u \in$  task_set via precedence_graph
4:   ready  $\leftarrow$  [ $t \mid t \in$  schedulable_order, in_deg[t] = 0]
5:   task_order  $\leftarrow$  []; visited  $\leftarrow$   $\emptyset$ 
6:   while ready  $\neq$   $\emptyset$  do
7:     pick  $\leftarrow$  random choice from ready
8:     ready.remove(pick)
9:     if pick  $\in$  visited then
10:      continue
```

```

11:     end if
12:     visited.add(pick); task_order.append(pick)
13:     for v ∈ precedence_graph[pick] do
14:         if v ∈ task_set then
15:             in_deg[v] -= 1
16:             if in_deg[v] = 0 then
17:                 ready.append(v)
18:             end if
19:         end if
20:     end for
21: end while
22: team_assignment ← [random choice(available_team[i]) | i ∈ task_order]
23: return Chromosome(task_order, team_assignment)
24: end function

```

---



---

#### Algorithm 11 Population Initialization

---

```

1: function INITIALIZE_POPULATION(population_size, schedulable_order, ...)
2:     population ← []
3:     population.append(GREEDY_CONSTRUCT(...))
4:     for i ← 1 to population_size - 1 do
5:         population.append(CONSTRUCT_RANDOM(schedulable_order,
        precedence_graph, available_team))
6:     end for
7:     return population
8: end function

```

---

### 5.2.3 Decode Chromosome and Fitness Function

The chromosome is decoded and evaluated using `DECODE_SOLUTION` and `CALC_FITNESS` defined in Section 5.1.4.

### 5.2.4 Selection Method

**Rank Selection** is used to choose parent chromosomes for reproduction. Instead of using raw fitness values, the population is first sorted by fitness. Then each individual receives a probability based on its rank.

Let  $n$  be the population size and  $i$  be the rank index, where  $i = 0$  is the best individual. The probability is assigned by a linear decay:

$$p_i = \frac{1}{n} \left( sp - \frac{2(sp-1)i}{n-1} \right)$$

In this project, we choose  $sp = 1.5$ . Therefore, the best individual receives probability  $\frac{1.5}{n}$ , while the worst individual receives probability  $\frac{0.5}{n}$ . This creates a linear decay of probabilities while still keeping:



$$\sum_{i=0}^{n-1} p_i = 1$$

Parent selection is then performed using roulette-wheel selection:

1. **Sorting:** Sort the population in ascending fitness order.
2. **Probability Assignment:** Assign each individual a linearly decayed probability based on its rank.
3. **Roulette-wheel Selection:** Randomly sample parents according to these probabilities, so better individuals are more likely to be selected but weaker individuals still have a chance.

---

**Algorithm 12** Rank Selection

---

```

1: function RANK_SELECTION(population, ...)
2:   Sort population ascending by CALC_FITNESS(·)
3:    $k \leftarrow |\text{population}|$ ;  $sp \leftarrow 1.5$ ;  $\text{probs} \leftarrow [0] \times k$ 
4:   for  $i \leftarrow 0$  to  $k - 1$  do
5:      $\text{probs}[i] \leftarrow \frac{1}{k} \left( sp - \frac{(2\,sp - 2)\,i}{k - 1} \right)$ 
6:   end for
7:   for  $i \leftarrow 1$  to  $k - 1$  do
8:      $\text{probs}[i] += \text{probs}[i - 1]$ 
9:   end for
10:   $\text{parents} \leftarrow []$ 
11:  for  $\_ \leftarrow 1$  to  $2$  do
12:     $r \leftarrow \text{random}()$ ;  $\text{chosen} \leftarrow \text{population}[-1]$ 
13:    for  $i \leftarrow 0$  to  $k - 1$  do
14:      if  $\text{probs}[i] \geq r$  then
15:         $\text{chosen} \leftarrow \text{population}[i]$ ; break
16:      end if
17:    end for
18:     $\text{parents.append}(\text{chosen})$ 
19:  end for
20:  return  $\text{parents}[0], \text{parents}[1]$ 
21: end function

```

---

### 5.2.5 Crossover Operator

The crossover operator is a key **genetic algorithm** mechanism that combines parts of two parent solutions to create new offspring. Inspired by biological reproduction, crossover aims to exploit good traits from both parents and pass them to the next generation.

- Take two parent chromosomes

- Mix their information to produce two more children
- Children inherit the strengths of both parents, increasing the chance of better solutions

We care about 2 main crossover:

1. Precedence-preserving Order-based Crossover (POX)
2. Crossover for Team Assignment

### Precedence-preserving Order-based Crossover

This crossover creates a new task order while trying to preserve the relative structure of both parents and maintain precedence feasibility.

#### Steps:

1. Initialize an empty child:
2. Traverse all positions in `parent1`. With probability 0.5, copy the task in that position into the same position as `the child`.
3. Traverse `parent2` from left to right and use its tasks to fill the remaining empty positions of `the child`, skipping duplicated tasks.
4. Check if the resulting child violates precedence constraints by Section 5.1.5, repair it using topological sorting.

---

#### Algorithm 13 Topological Repair

---

```

1: function TOPOLOGICAL_REPAIR(task_order, precedence_graph)
2:   task_set  $\leftarrow$  set(task_order)
3:   Compute in_deg[u] for all  $u \in$  task_set via precedence_graph
4:   queue  $\leftarrow$  deque( $\{t \mid t \in$  task_order, in_deg[t] = 0 $\}$ )
5:   result  $\leftarrow$  []; visited  $\leftarrow$   $\emptyset$ 
6:   while queue is not empty do
7:      $u \leftarrow$  queue.popleft()
8:     if  $u \in$  visited then
9:       continue
10:    end if
11:    visited.add( $u$ ); result.append( $u$ )
12:    for  $v \in$  precedence_graph[ $u$ ] do
13:      if  $v \in$  task_set then
14:        in_deg[v]  $-=$  1
15:        if in_deg[v] = 0 then
16:          queue.append( $v$ )
17:        end if

```

```

18:         end if
19:     end for
20: end while
21: return result
22: end function

```

---



---

**Algorithm 14** Precedence-preserving Order-based Crossover (POX)

---

```

1: function POX(parent1, parent2, precedence, precedence_graph)
2:   child  $\leftarrow [-1] \times |\text{parent1}|$ ;   added  $\leftarrow \emptyset$ 
3:   for  $i \leftarrow 0$  to  $|\text{parent1}| - 1$  do
4:     if  $\text{random}() \geq 0.5$  then
5:       child[i]  $\leftarrow$  parent1[i];   added.add(parent1[i])
6:     end if
7:   end for
8:   j  $\leftarrow 0$ 
9:   for  $i \leftarrow 0$  to  $|\text{child}| - 1$  do
10:    if child[i] = -1 then
11:      while parent2[j]  $\in$  added do
12:        j += 1
13:      end while
14:      child[i]  $\leftarrow$  parent2[j];   added.add(parent2[j]);   j += 1
15:    end if
16:  end for
17:  if  $\neg \text{Is\_PRECEDENCE\_FEASIBLE}(\text{child}, \text{precedence})$  then
18:    child  $\leftarrow$  TOPOLOGICAL_REPAIR(child, precedence_graph)
19:  end if
20:  return child
21: end function

```

---

**Team Assignment Crossover (TAX)**

The goal of **Team Assignment Crossover** is to:

- Inherit good cost-efficient or feasible team assignments from both parents
- Create diversity in the population
- Potentially reduce total cost or improve task feasibility

At each step, randomly choose the team assignment from one of the two parents:

child\_team[i]  $\leftarrow$  random\_choice([parent1\_team[i], parent2\_team[i]])

More specifically, for each task in the child's order, the team is chosen randomly from the two parent assignments identified by task identity. If neither parent provides a valid team, a random capable team is assigned.

---

**Algorithm 15** Team Assignment Crossover (TAX)

---

```

1: function TAX(task_order1, team1, task_order2, team2, available_team,
   child_task_order)
2:   map1 ← {task : team | (task, team) ∈ zip(task_order1, team1)}
3:   map2 ← {task : team | (task, team) ∈ zip(task_order2, team2)}
4:   child_team ← []
5:   for currentTask ∈ child_task_order do
6:     possible ← []
7:     if currentTask ∈ map1 and
       map1[currentTask] ∈ available_team[currentTask] then
8:       possible.append(map1[currentTask])
9:     end if
10:    if currentTask ∈ map2 and
       map2[currentTask] ∈ available_team[currentTask] then
11:      possible.append(map2[currentTask])
12:    end if
13:    if |possible| > 0 then
14:      child_team.append(random choice from possible)
15:    else
16:      child_team.append(random choice from available_team[currentTask])
17:    end if
18:  end for
19:  return child_team
20: end function

```

---

## Offspring Generation

Two children are produced per crossover event by applying POX and TAX symmetrically on both parents.

---

### Algorithm 16 Offspring Generation

---

```

1: function OFFSPRING_GENERATE(parent1, parent2, precedence,
   precedence_graph, available_team)
2:   to1, tm1 ← parent1.task_order, parent1.team_assignment
3:   to2, tm2 ← parent2.task_order, parent2.team_assignment
4:   child_task1 ← POX(to1, to2, precedence, precedence_graph)
5:   child_task2 ← POX(to2, to1, precedence, precedence_graph)
6:   child_team1 ← TAX(to1, tm1, to2, tm2, available_team, child_task1)
7:   child_team2 ← TAX(to2, tm2, to1, tm1, available_team, child_task2)
8:   return Chromosome(child_task1, child_team1), Chromosome(child_task2,
   child_team2)
9: end function

```

---

## 5.2.6 Mutation

Mutation introduces small, random changes to the offspring. For example:

- Changing the team assigned to a task

- Swapping two tasks in the order, if the new order is still valid

This improves the search in many ways:

1. Maintains genetic diversity
2. Avoids local optima
3. Explores the search space more thoroughly
4. Improves robustness

Two main mutations are used in this project:

- **Task Order Mutation:** Swap two tasks in the task order. The mutation is accepted only if the resulting order still satisfies all precedence constraints.
- **Team Assignment Mutation:** Randomly reassign a task to another valid team that is capable of performing that task.

---

#### Algorithm 17 Mutation

---

```

1: function MUTATE(ind, precedence, available_team, attempts, mutation_rate)
2:   task_order, team_assignment  $\leftarrow$  ind.task_order[:], ind.team_assignment[:]
3:   new_task_order  $\leftarrow$  TASK_ORDER_MUTATION(task_order,
4:     team_assignment, precedence, attempts)
5:   new_team_assignment  $\leftarrow$  TEAM_ASSIGNMENT_MUTATION(new_task_order,
6:     team_assignment, available_team, mutation_rate)
7:   return Chromosome(new_task_order, new_team_assignment)
8: end function

```

---

### 5.2.7 Evaluate and Replace

This is when we evaluate the new offspring and replace them into the population. This step flow is as follows:

1. Append the offspring to the current population.
2. Evaluate and sort the population according to the lexicographic fitness 5.1.4:
3. Only keep the first `population_size` individuals of the population.

---

#### Algorithm 18 Evaluate and Replace

---

```

1: function EVALUATE_AND_REPLACE(population, child1, child2, ...)
2:   population_size  $\leftarrow$  |population|
3:   population.extend([child1, child2])
4:   Sort population ascending by CALC_FITNESS(·)
5:   return population[: population_size]
6: end function

```

---

### 5.2.8 Termination Condition

The algorithm terminates when the elapsed wall-clock time reaches  $\text{max\_time} = 295$  seconds.

### 5.2.9 Main Loop

---

**Algorithm 19** Genetic Algorithm — Main Loop

---

```
1: function GA_MAIN
2:   start  $\leftarrow$  time.time();   max_time  $\leftarrow$  295;   population_size  $\leftarrow$  10
3:   population  $\leftarrow$  INITIALIZE_POPULATION(population_size, ...)
4:   while True do
5:     if time.time() - start  $\geq$  max_time then
6:       break
7:     end if
8:     parent1, parent2  $\leftarrow$  RANK_SELECTION(population, ...)
9:     child1, child2  $\leftarrow$  OFFSPRING_GENERATE(parent1, parent2, ...)
10:    child1  $\leftarrow$  MUTATE(child1, precedence, available_team, 5, 0.05)
11:    child2  $\leftarrow$  MUTATE(child2, precedence, available_team, 5, 0.05)
12:    population  $\leftarrow$  EVALUATE_AND_REPLACE(population, child1, child2, ...)
13:  end while
14:  return population[0]
15: end function
```

---

## 5.3 Simulated Annealing

**Introduction:** Simulated Annealing (SA) is a meta-heuristic inspired by the physical annealing process in metallurgy, where a material is heated and then slowly cooled to reduce defects and reach a stable low-energy state.

**Main Idea:** This method balances exploration (accepting bad moves early on) and exploitation (refining good solutions later) by controlling a *temperature* parameter over time. In optimization, SA searches for a good solution by:

- Starting from a random solution
- Making small random changes, called perturbations
- Accepting changes that improve the solution
- Occasionally accepting worse solutions to escape local optima
- Gradually reducing the chance of accepting worse solutions by cooling

Since SA shares the solution representation, greedy initialization, decode, fitness, and neighbor generation components with GA (Section 5.1), only the SA-specific components are described below.

### 5.3.1 State Representation

The state representation of SA is identical to the solution representation defined in Section 5.1.2 (i.e. the same chromosome structure used by GA).

```
State: {  
    "task_order"      : [3, 1, 4, 2, ..., 5],  
    "team_assignment" : [2, 1, 3, 1, ..., 2]  
}
```

Unlike GA, SA works with a single current state at a time rather than a population.

### 5.3.2 Create Initial Feasible Solution

SA starts from the greedy solution produced by GREEDY\_CONSTRUCT defined in Section 5.1.3.

### 5.3.3 Decode State and Fitness Function

Identical to Section 5.1.4. The same lexicographic fitness is used:

$$\text{fitness} = (\text{num\_unscheduled\_task}, \text{completion\_time}, \text{total\_cost})$$

### 5.3.4 Neighbor Generation

A neighbor is produced by applying TASK\_ORDER\_MUTATION and TEAM\_ASSIGNMENT\_MUTATION in sequence, both defined in Section 5.1.6.

---

**Algorithm 20** Get Neighbor (SA)

---

```
1: function GET_NEIGHBOR(curr, precedence, available_team, attempts,  
    mutation_rate)  
2:   task_order, team_assignment  $\leftarrow$  curr.task_order[:], curr.team_assignment[:]  
3:   new_task_order  $\leftarrow$  TASK_ORDER_MUTATION(task_order,  
    team_assignment, precedence, attempts)  
4:  
   new_team_assignment  $\leftarrow$  TEAM_ASSIGNMENT_MUTATION(new_task_order,  
    team_assignment, available_team, mutation_rate)  
5:   return State(new_task_order, new_team_assignment)  
6: end function
```

---

### 5.3.5 Acceptance Probability

The acceptance rule has two cases:

1. If the neighbor solution is no worse than the current solution:

$$f_{\text{neigh}} \leq f_{\text{curr}}$$

then the neighbor is always accepted.

2. Otherwise, the neighbor is worse. The algorithm finds the first objective where the two fitness tuples differ, then computes:

$$\Delta = \frac{b - a}{\max(1.0, |a|)}$$

where  $a$  is the current objective value and  $b$  is the neighbor objective value. The division by  $\max(1.0, |a|)$  normalizes the increase because different objectives have different units and scales.

The worse neighbor is accepted with probability:

$$p = \exp\left(-\frac{\Delta}{T}\right)$$

A high temperature  $T$  makes worse solutions easier to accept, while a low temperature makes them harder to accept.

---

**Algorithm 21** Acceptance Criterion (Metropolis)

---

```

1: function ACCEPTANCE( $f_{\text{curr}}, f_{\text{neigh}}, T$ )
2:   if  $f_{\text{neigh}} \leq f_{\text{curr}}$  then
3:     return True
4:   end if
5:   for  $(a, b) \in \text{zip}(f_{\text{curr}}, f_{\text{neigh}})$  do
6:     if  $a \neq b$  then
7:        $\Delta \leftarrow \frac{b - a}{\max(1.0, |a|)}$ ; break
8:     end if
9:   end for
10:   $p \leftarrow \exp\left(-\frac{\Delta}{T}\right)$ 
11:  return  $\text{random}() \leq p$ 
12: end function

```

---

### 5.3.6 Termination Condition

Identical to GA: the algorithm stops when elapsed time reaches  $\text{max\_time} = 295$  seconds.

### 5.3.7 Cooling Schedule and Main Loop

A linear cooling schedule is used:

$$T(t) = \max\left(T_{\min}, T_0 \times \left(1 - \frac{t}{\text{max\_time}}\right)\right)$$



where  $T_0 = 50.0$  and  $T_{\min} = 10^{-3}$ . The temperature is recomputed from elapsed time at every iteration, so it decreases monotonically without needing an explicit multiplicative step.

---

**Algorithm 22** Simulated Annealing — Main Loop

---

```

1: function SA_MAIN
2:   start  $\leftarrow$  time.time();   max_time  $\leftarrow$  295;    $T_0 \leftarrow 50.0$ ;    $T_{\min} \leftarrow 10^{-3}$ 
3:   curr  $\leftarrow$  GREEDY_CONSTRUCT(...)
4:   best  $\leftarrow$  curr;    $f_{\text{curr}} \leftarrow$  CALC_FITNESS(curr, ...);    $f_{\text{best}} \leftarrow f_{\text{curr}}$ 
5:   while True do
6:     elapsed  $\leftarrow$  time.time() - start
7:     if elapsed  $\geq$  max_time then
8:       break
9:     end if
10:     $T \leftarrow \max(T_{\min}, T_0 \times (1 - \text{elapsed}/\text{max\_time}))$ 
11:    neighbor  $\leftarrow$  GET_NEIGHBOR(curr, precedence, available_team, 5, 0.1)
12:     $f_{\text{neigh}} \leftarrow$  CALC_FITNESS(neighbor, ...)
13:    if ACCEPTANCE( $f_{\text{curr}}, f_{\text{neigh}}, T$ ) then
14:      curr,  $f_{\text{curr}} \leftarrow$  neighbor,  $f_{\text{neigh}}$ 
15:      if  $f_{\text{neigh}} < f_{\text{best}}$  then
16:        best,  $f_{\text{best}} \leftarrow$  neighbor,  $f_{\text{neigh}}$ 
17:      end if
18:    end if
19:  end while
20:  return best
21: end function

```

---

## 5.4 Tabu Search

**Introduction:** Tabu Search (TS) enhances local search by incorporating a memory structure *the tabu list* to prevent cycling and facilitate escape from local optima. At each iteration, the best admissible neighbor is selected, even if it is worse than the current solution.

**Main Idea:** TS inherits the greedy selection of hill climbing by always choosing the best neighbor from a generated neighbor list. The difference lies in the tabu list, which stores recent moves to prevent immediately undoing them:

- The tabu list records recently performed moves, marking them as temporarily forbidden.
- The duration a move stays forbidden is called the *tabu tenure* (measured in iterations).
- This helps the algorithm avoid cycling, escape local optima, and maintain search diversity.

### 5.4.1 State and Move Representation

The state representation of Tabu Search is identical to the solution representation defined in Section 5.1.2. Therefore, each state still consists of a `task_order` and a `team_assignment`.

The main difference is that Tabu Search additionally records the move used to generate a neighbor. In this implementation, a move is represented as:

$$\text{move} = (\text{swap\_key}, \text{team\_changes})$$

where:

- `swap_key` records the pair of swapped tasks:

$$\text{swap\_key} = \text{tuple}(\text{sorted}((\text{task\_i}, \text{task\_j})))$$

If no valid swap is performed, `swap_key = None`.

- `team_changes` records all team reassignment changes:

$$\text{team\_changes} = ((\text{task}, \text{old\_team}, \text{new\_team}), \dots)$$

If no team assignment is changed, it is an empty tuple.

This representation is compact because the tabu list stores only recent moves, not entire solutions.

### 5.4.2 Create Initial Feasible State

Identical to SA: the search starts from the greedy solution produced by `GREEDY_CONSTRUCT` define in Section 5.1.3.

### 5.4.3 Decode State and Fitness Calculation

Identical to Section 5.1.4.

### 5.4.4 Neighbor Generation

The TS variant of the neighbor generation extends the shared mutation operators to also return move-tracking information required by the tabu mechanism.

---

**Algorithm 23** Task Order Mutation with Move Tracking (TS)

---

```
1: function TASK_ORDER_MUTATION_TS(task_order, team_assignment,
   precedence, attempts)
2:   if |task_order| < 2 then
3:     return task_order[:], team_assignment[:], None
```

```

4:   end if
5:   for  $k \leftarrow 1$  to attempts do
6:       new_order  $\leftarrow$  task_order[:]
7:        $i, j \leftarrow$  random sample of 2 distinct indices
8:       swap(new_order[ $i$ ], new_order[ $j$ ])
9:       if IS_PRECEDENCE_FEASIBLE(new_order, precedence) then
10:          new_team  $\leftarrow$  team_assignment[:]
11:          swap(new_team[ $i$ ], new_team[ $j$ ])
12:          swap_key  $\leftarrow$  tuple(sorted({task_order[ $i$ ], task_order[ $j$ ]}))
13:          return new_order, new_team, swap_key
14:       end if
15:   end for
16:   return task_order[:], team_assignment[:], None
17: end function

```

---



---

**Algorithm 24** Team Assignment Mutation with Change Tracking (TS)

---

```

1: function TEAM_ASSIGNMENT_MUTATION_TS(task_order, team_assignment,
   available_team, mutation_rate)
2:   new_assignment  $\leftarrow$  team_assignment[:]; changes  $\leftarrow$  []
3:   for (idx, task)  $\in$  enumerate(task_order) do
4:       if random() < mutation_rate then
5:           viable  $\leftarrow$  available_team[task]
6:           if |viable| > 1 then
7:               new_team  $\leftarrow$  random choice from viable
8:               if new_team  $\neq$  new_assignment[idx] then
9:                   changes.append((task, new_assignment[idx], new_team))
10:                  new_assignment[idx]  $\leftarrow$  new_team
11:               end if
12:           end if
13:       end if
14:   end for
15:   return new_assignment, changes
16: end function

```

---



---

**Algorithm 25** Get Neighbor with Move (TS)

---

```

1: function GET_NEIGHBOR_TS(curr, precedence, available_team, attempts,
   mutation_rate)
2:   new_to, team_swap,
   swap_key  $\leftarrow$  TASK_ORDER_MUTATION_TS(curr.task_order,
   curr.team_assignment, precedence, attempts)
3:   new_tm, team_changes  $\leftarrow$  TEAM_ASSIGNMENT_MUTATION_TS(new_to,
   team_swap, available_team, mutation_rate)
4:   move  $\leftarrow$  (swap_key, tuple(team_changes))
5:   return State(new_to, new_tm), move
6: end function

```

---

### 5.4.5 Move Identification and Tabu Check

This is the main difference of Tabu Search. After a neighbor is generated, the algorithm identifies the move that produced it and checks whether the move is admissible.

A move is considered **admissible** in one of two cases:

- **Not tabu:** the move is not in the tabu list, so applying it does not immediately cycle back to a recently visited solution.
- **Aspiration criterion:** even if the move is tabu, it is still allowed if the resulting solution improves the current global best solution.

Therefore, the tabu list prevents cycling, while the aspiration criterion allows the search to accept a very good move even when it is temporarily tabu.

---

**Algorithm 26** Tabu Check

---

```
1: function IS_TABU(move, tabu_swap, tabu_team, gen)
2:   swap_key, team_changes  $\leftarrow$  move
3:   if swap_key  $\neq$  None then
4:     expire  $\leftarrow$  tabu_swap.get(swap_key)
5:     if expire  $\neq$  None and expire > gen then
6:       return True
7:     end if
8:   end if
9:   for (task, old_team, new_team)  $\in$  team_changes do
10:    expire  $\leftarrow$  tabu_team.get((task, new_team))
11:    if expire  $\neq$  None and expire > gen then
12:      return True
13:    end if
14:  end for
15:  return False
16: end function
```

---

---

**Algorithm 27** Tabu Registration

---

```
1: function REGISTER_TABU(move, tabu_swap, tabu_team, gen, tenure)
2:   swap_key, team_changes  $\leftarrow$  move
3:   if swap_key  $\neq$  None then
4:     tabu_swap[swap_key]  $\leftarrow$  gen + tenure
5:   end if
6:   for (task, old_team, new_team)  $\in$  team_changes do
7:     tabu_team[(task, old_team)]  $\leftarrow$  gen + tenure
8:   end for
9: end function
```

---

### 5.4.6 Termination Condition

The algorithm terminates when elapsed time reaches  $\text{max\_time} = 295$  seconds. Expired tabu entries are pruned every 200 generations to manage memory.

### 5.4.7 Main Loop

---

**Algorithm 28** Tabu Search — Main Loop

---

```
1: function TS_MAIN
2:   start  $\leftarrow$  time.time();   max_time  $\leftarrow$  295;   num_neighbors  $\leftarrow$ 
   10;   tabu_tenure  $\leftarrow$  10
3:   tabu_swap  $\leftarrow$  {};   tabu_team  $\leftarrow$  {}
4:   curr  $\leftarrow$  GREEDY_CONSTRUCT(...);   f_curr  $\leftarrow$  CALC_FITNESS(curr, ...)
5:   best  $\leftarrow$  curr;   f_best  $\leftarrow$  f_curr;   gen  $\leftarrow$  0
6:   while True do
7:     if time.time() - start  $\geq$  max_time then
8:       break
9:     end if
10:
   gen += 1;   best_candidate  $\leftarrow$  None;   f_bc  $\leftarrow$  None;   best_move  $\leftarrow$  None
11:   for _  $\leftarrow$  1 to num_neighbors do
12:     neighbor, move  $\leftarrow$  GET_NEIGHBOR_TS(curr, precedence,
   available_team, 5, 0.1)
13:     fit  $\leftarrow$  CALC_FITNESS(neighbor, ...)
14:
   admissible  $\leftarrow$   $\neg$ IS_TABU(move, tabu_swap, tabu_team, gen) or (fit < f_best)
15:     if admissible and (f_bc = None or fit < f_bc) then
16:       best_candidate, f_bc, best_move  $\leftarrow$  neighbor, fit, move
17:     end if
18:   end for
19:   if best_candidate = None then
20:     continue
21:   end if
22:   curr, f_curr  $\leftarrow$  best_candidate, f_bc
23:   if f_curr < f_best then
24:     best, f_best  $\leftarrow$  curr, f_curr
25:   end if
26:   REGISTER_TABU(best_move, tabu_swap, tabu_team, gen, tabu_tenure)
27:   if gen mod 200 = 0 then
28:     Remove entries from tabu_swap and tabu_team where expire  $\leq$  gen
29:   end if
30: end while
31: return best
32: end function
```

---

## 5.5 Ant Colony Optimization

**Introduction:** Ant Colony Optimization (ACO) is a population-based metaheuristic inspired by the foraging behavior of real ants. In nature, ants deposit pheromone on paths leading to food sources. Paths with stronger pheromone are more likely to be followed by later ants, so good paths are gradually reinforced over time.

**Main Idea:** In this project, each ant constructs a solution by assigning a feasible team to each task. The team choice is probabilistic and depends on two main factors: *pheromone intensity*, which represents learned experience from previous good solutions, and *heuristic desirability*, which estimates how promising a team assignment is at the current step.

The general procedure of ACO is:

1. Initialize pheromone values for all valid task–team assignments.
2. Let multiple ants construct candidate solutions.
3. Evaluate all constructed solutions using the common fitness function.
4. Evaporate pheromone globally and reinforce good assignments.
5. Repeat until the time limit is reached.

### 5.5.1 State Representation

ACO uses the same state representation as the other metaheuristic methods. As defined in Section 5.1.2, each solution consists of two components:

$$\text{solution} = (\text{task\_order}, \text{team\_assignment})$$

In ACO, each ant builds one such solution. The `task_order` is the common schedulable order, while `team_assignment` is constructed step by step according to pheromone and heuristic information.

### 5.5.2 Create Initial Feasible Solution

ACO uses the same greedy construction procedure described in Section 5.1.3 to create an initial feasible solution.

This greedy solution is used as a seed for the search. It provides an initial best solution before the ants start exploring alternative assignments.

$$\text{best\_solution} \leftarrow \text{Greedy\_Construct}()$$

After that, ACO improves the solution by repeatedly generating new candidate solutions through ant construction and pheromone updates.

### 5.5.3 Decode State and Fitness Function

ACO also uses the same decoding and fitness evaluation mechanism described in Section 5.1.4.

Therefore, ACO does not define a new fitness function. Its main difference lies in how new team assignments are constructed and how pheromone values are updated.

### 5.5.4 Pheromone Representation and Initialization

Pheromone is stored as a nested dictionary  $\tau[\text{task}][\text{team}]$ , representing the learned desirability of assigning **team** to **task**. All entries are initialised to  $\tau_0 = 1.0$ . The pheromone trails are bounded between  $\tau_{\min}$  and  $\tau_{\max}$  following the Max–Min Ant System (MMAS) variant.

---

#### Algorithm 29 Pheromone Initialization

---

```

1: function INIT_PHEROMONE(schedulable_order, available_team)
2:    $\tau \leftarrow \{\}$ 
3:   for task  $\in$  schedulable_order do
4:      $\tau[\text{task}] \leftarrow \{\text{team} : \tau_0 \mid \text{team} \in \text{available\_team}[\text{task}]\}$ 
5:   end for
6:   return  $\tau$ 
7: end function

```

---

### 5.5.5 Ant Solution Construction

This is the core step of Ant Colony Optimization. Each ant constructs one complete solution by processing tasks in the fixed schedulable order. At each task, the ant chooses one feasible team using **roulette-wheel selection**.

For a current task, each candidate team is assigned a selection weight:

$$w(\text{task}, \text{team}) = \tau[\text{task}][\text{team}]^\alpha \times \eta(\text{task}, \text{team})^\beta$$

where:

- $\tau[\text{task}][\text{team}]$  is the pheromone value of assigning that team to the task. A larger pheromone value means this assignment has appeared in good previous solutions.
- $\eta(\text{task}, \text{team})$  is the heuristic desirability of the assignment:

$$\eta(\text{task}, \text{team}) = \frac{1}{\text{estimated finish time} + 1}$$

A team with an earlier estimated finish time has a larger heuristic value.

- $\alpha$  controls the influence of pheromone information.
- $\beta$  controls the influence of heuristic information.

After computing all candidate weights, the probability of choosing a team is:

$$P(\text{team}) = \frac{w(\text{task}, \text{team})}{\sum_{k \in \text{candidate teams}} w(\text{task}, k)}$$

Therefore, roulette-wheel selection works as follows:

- teams with larger weights occupy larger portions of the roulette wheel;
- a random value is sampled from the total weight;
- the selected team is the first one whose cumulative weight reaches that random value.

This mechanism balances two goals: exploiting assignments with strong pheromone trails and exploring assignments that look promising according to the current schedule.

---

**Algorithm 30** Ant Solution Construction

---

```

1: function ANT_CONSTRUCT(schedulable_order, rev_graph, available_team,
   duration, team_avail,  $\tau$ ,  $\alpha$ ,  $\beta$ )
2:   task_End  $\leftarrow$   $[0] \times \text{num\_task}$ ;   teamAvail  $\leftarrow$  team_avail[:]
3:   task_order  $\leftarrow$  [];   team_assignment  $\leftarrow$  []
4:   for currentTask  $\in$  schedulable_order do
5:     pred_finish  $\leftarrow$  max(task_End[p] |  $p \in \text{rev\_graph}[\text{currentTask}]$ )
6:     teams  $\leftarrow$  available_team[currentTask];   weights  $\leftarrow$  []
7:     for team  $\in$  teams do
8:       earliest  $\leftarrow$  max(teamAvail[team], pred_finish)
9:       finish  $\leftarrow$  earliest + duration[currentTask]
10:       $\eta \leftarrow 1.0/(\text{finish} + 1)$ 
11:      weights.append( $\tau[\text{currentTask}][\text{team}]^\alpha \times \eta^\beta$ )
12:   end for
13:   total  $\leftarrow$   $\sum$  weights
14:   if total = 0 then
15:     chosen  $\leftarrow$  random choice from teams
16:   else
17:      $r \leftarrow \text{random}() \times \text{total}$ ;   cumulative  $\leftarrow$  0;   chosen  $\leftarrow$  teams[-1]
18:     for (team, w)  $\in$  zip(teams, weights) do
19:       cumulative += w
20:       if cumulative  $\geq r$  then
21:         chosen  $\leftarrow$  team;   break
22:       end if
23:     end for
24:   end if
25:   earliest  $\leftarrow$  max(teamAvail[chosen], pred_finish)
26:   task_End[currentTask]  $\leftarrow$  earliest + duration[currentTask]
27:   teamAvail[chosen]  $\leftarrow$  task_End[currentTask]
28:   task_order.append(currentTask);   team_assignment.append(chosen)

```



```

29:   end for
30:   return Solution(task_order, team_assignment)
31: end function

```

---

### 5.5.6 Pheromone Update

After each iteration, ACO updates pheromone values in two main steps:

- **Evaporation:** decrease all pheromone values to reduce the influence of old search information.
- **Deposit:** add pheromone to good task–team assignments found by the global best and the iteration best solutions.

All pheromone values are kept within the range  $[\tau_{\min}, \tau_{\max}]$  to avoid values becoming too small or too large.

#### Evaporation

- Every pheromone value is multiplied by  $(1 - \rho)$ .
- The lower bound  $\tau_{\min}$  ensures that each valid assignment still has a chance to be selected in later iterations.

---

#### Algorithm 31 Pheromone Evaporation

---

```

1: function EVAPORATE( $\tau$ ,  $\rho$ )
2:   for task  $\in \tau$  do
3:     for team  $\in \tau[\text{task}]$  do
4:        $\tau[\text{task}][\text{team}] \leftarrow \max(\tau_{\min}, \tau[\text{task}][\text{team}] \times (1 - \rho))$ 
5:     end for
6:   end for
7: end function

```

---

#### Deposit

- Good solutions deposit additional pheromone on the assignments they use.
- A better solution receives a larger pheromone contribution.
- The upper bound  $\tau_{\max}$  prevents one assignment from dominating the search too early.

---

#### Algorithm 32 Pheromone Deposit

---

```

1: function DEPOSIT( $\tau$ , sol, fitness)
2:   ( $\cdot$ , completion,  $\cdot$ )  $\leftarrow$  fitness
3:    $\delta \leftarrow 1.0 / (\text{completion} + 1)$ 
4:   for (task, team)  $\in$  zip(sol.task_order, sol.team_assignment) do

```

---

```

5:       $\tau[\text{task}][\text{team}] \leftarrow \min(\tau_{\max}, \tau[\text{task}][\text{team}] + \delta)$ 
6:    end for
7: end function

```

---

### 5.5.7 Termination Condition

The algorithm terminates when elapsed time reaches  $\text{max\_time} = 295$  seconds.

### 5.5.8 Main Loop

ACO parameters:  $\text{num\_ants} = 10$ ,  $\alpha = 1.0$ ,  $\beta = 2.0$ ,  $\rho = 0.1$ ,  $\tau_0 = 1.0$ ,  $\tau_{\min} = 10^{-4}$ ,  $\tau_{\max} = 10.0$ . The global best and iteration best both deposit pheromone each round to balance exploitation and exploration.

---

#### Algorithm 33 Ant Colony Optimization — Main Loop

---

```

1: function ACO_MAIN
2:   start  $\leftarrow$  time.time();   max_time  $\leftarrow$  295
3:    $\tau \leftarrow$  INIT_PHEROMONE(schedulable_order, available_team)
4:   best  $\leftarrow$  GREEDY_CONSTRUCT(...);    $f_{\text{best}} \leftarrow$  CALC_FITNESS(best, ...)
5:   DEPOSIT( $\tau$ , best,  $f_{\text{best}}$ )
6:   while True do
7:     elapsed  $\leftarrow$  time.time() - start
8:     if elapsed  $\geq$  max_time then
9:       break
10:    end if
11:    iter_best  $\leftarrow$  None;    $f_{\text{iter}} \leftarrow$  None
12:    for  $\_ \leftarrow 1$  to num_ants do
13:      ant_sol  $\leftarrow$  ANT_CONSTRUCT(...,  $\tau$ ,  $\alpha$ ,  $\beta$ )
14:       $f_{\text{ant}} \leftarrow$  CALC_FITNESS(ant_sol, ...)
15:      if  $f_{\text{iter}} = \text{None}$  or  $f_{\text{ant}} < f_{\text{iter}}$  then
16:        iter_best,  $f_{\text{iter}} \leftarrow$  ant_sol,  $f_{\text{ant}}$ 
17:      end if
18:      if  $f_{\text{ant}} < f_{\text{best}}$  then
19:        best,  $f_{\text{best}} \leftarrow$  ant_sol,  $f_{\text{ant}}$ 
20:      end if
21:    end for
22:    EVAPORATE( $\tau$ ,  $\rho$ )
23:    DEPOSIT( $\tau$ , best,  $f_{\text{best}}$ )
24:    if iter_best  $\neq$  None then
25:      DEPOSIT( $\tau$ , iter_best,  $f_{\text{iter}}$ )
26:    end if
27:  end while
28:  return best
29: end function

```

---

## 5.6 Results

This section compares the judge scores obtained by the four implemented metaheuristic algorithms: Genetic Algorithm (GA), Ant Colony Optimization (ACO), Simulated Annealing (SA), and Tabu Search (TS).

Each result is reported as:

(tasks scheduled, completion time, total cost)

This corresponds to the fitness criteria in Section 5.1.4. For readability, the table shows the number of scheduled tasks directly, while the internal fitness minimizes the number of unscheduled tasks.

**Comparison rule.** All algorithms are compared lexicographically:

more scheduled tasks  $\rightarrow$  smaller completion time  $\rightarrow$  lower total cost.

In this dataset, all methods schedule the same number of tasks as the jury, so completion time and total cost become the main differentiators.

### 5.6.1 Score Comparison

Table 5.1: Jury reference score versus GA, ACO, SA, and TS on ten test cases. Each cell is written as (tasks scheduled, completion time, total cost). Bold entries indicate the best algorithmic result in each row; ties are bolded together.

| No. | Tasks | Teams | Prec. | Jury              | GA                      | ACO                      | SA                       | TS                    |
|-----|-------|-------|-------|-------------------|-------------------------|--------------------------|--------------------------|-----------------------|
| 1   | 5     | 6     | 4     | (5, 235, 145)     | <b>(5, 235, 145)</b>    | <b>(5, 235, 145)</b>     | <b>(5, 235, 145)</b>     | <b>(5, 235, 145)</b>  |
| 2   | 10    | 15    | 6     | (10, 210, 430)    | (10, 210, 350)          | (10, 190, 530)           | <b>(10, 190, 480)</b>    | <b>(10, 190, 480)</b> |
| 3   | 100   | 80    | 100   | (100, 550, 4510)  | <b>(100, 550, 1920)</b> | (100, 550, 2090)         | (100, 550, 3050)         | (100, 550, 4510)      |
| 4   | 200   | 80    | 150   | (200, 370, 13240) | (200, 360, 8880)        | (200, 360, 7140)         | <b>(200, 350, 14950)</b> | (200, 370, 13240)     |
| 5   | 500   | 200   | 650   | (493, 840, 38150) | (493, 780, 32430)       | <b>(493, 780, 28680)</b> | (493, 780, 40210)        | (493, 780, 39930)     |
| 6   | 600   | 300   | 750   | (599, 550, 42300) | (599, 530, 41210)       | <b>(599, 530, 32210)</b> | (599, 530, 55540)        | (599, 550, 42300)     |
| 7   | 700   | 300   | 750   | (657, 510, 46270) | (657, 450, 50060)       | <b>(657, 450, 34800)</b> | (657, 490, 63890)        | (657, 510, 46270)     |
| 8   | 800   | 400   | 750   | (786, 530, 63960) | (786, 520, 60040)       | <b>(786, 520, 50330)</b> | (786, 520, 72680)        | (786, 520, 81880)     |
| 9   | 900   | 400   | 750   | (884, 820, 74300) | (884, 710, 66090)       | <b>(884, 710, 58660)</b> | (884, 710, 77510)        | (884, 710, 83740)     |
| 10  | 1000  | 400   | 1750  | (980, 770, 68520) | (980, 760, 72510)       | <b>(980, 760, 62150)</b> | (980, 760, 95070)        | (980, 770, 68520)     |

*Note.* All four algorithms use the same wall-clock budget of approximately 295 seconds. Therefore, runtime is not shown in the table because it does not distinguish the methods.

### 5.6.2 Observations

- **All methods match the jury in scheduled-task count.** This means the search mainly affects completion time and total cost, rather than the number of feasible tasks.

- **Small instances are easy for all methods.** On the smallest test case, all algorithms obtain exactly the same score. On the second test case, SA and TS even improve the jury completion time from 210 to 190.
- **ACO performs best on large instances.** From  $N = 500$  to  $N = 1000$ , ACO gives the best result in every test case. Its advantage mainly comes from lower total cost while keeping the same completion time as the other strong methods.
- **GA is competitive on medium-sized cases.** GA achieves the best result for  $N = 100$  and remains close to ACO in several larger cases, but it is usually more expensive than ACO.
- **SA and TS are strong in completion time but weaker in cost.** These methods often reach good makespan values, but their team assignments tend to have higher total cost on larger instances.
- **Overall, ACO is the most stable method in this experiment.** Since ACO focuses directly on task–team assignment through pheromone learning, it exploits cost-efficient assignments better under the same time limit.